

Unifying the many ways to compare

Date: 2018-10-07

Author: David Stone (davidmstone@google.com, david@doublewise.net)

Audience: Library Evolution Working Group (LEWG)

Dependencies

- [P0790: Effect of operator<=> on the C++ Standard Library](#)
- [P1190: I did not order this](#)

Background

We currently have several facilities to compare things (mostly strings). We should unify on "the one true way". For reference, the currently existing functionality relevant to this paper (other than existing comparison operators) is:

- `char_traits::eq` (returns bool)
- `char_traits::eq_int_type` (returns bool)
- `char_traits::lt` (returns bool)
- `char_traits::compare` (returns int)
- `basic_string::compare` (returns int)
- `basic_string_view::compare` (returns int)
- `sub_match::compare` (returns int)
- `istreambuf_iterator::equal` (returns bool)
- `filesystem::path::compare` (returns int)
- `filesystem::equivalent` (returns bool, provides the weak equality of whether two paths resolve to the same file)

Note that for `char_traits`, we also have a "character traits" 'concept' with the same requirements as the concrete `char_traits` type.

Proposal

- Add `char_traits::cmp`, which returns `std::strong_ordering` for all built-in character types.
- Require all character traits classes to
 - have a static member function `cmp` that returns a comparison category or
 - have `lt` and `compare` (as they do now).
- Deprecate support for character traits classes that do not have static member `cmp`.
- Remove the requirement that character traits classes provide `lt` and `compare`.
- Deprecate `char_traits::lt` and `char_traits::compare`.
 - Note: we may also deprecate `eq` and `eq_int_type`, depending on the outcome of [P1190: I did not order this](#).
- Deprecate `basic_string::compare`, `basic_string_view::compare`, `sub_match::compare`, and `filesystem::path::compare` in favor of `operator<=>`.
- Deprecate `istreambuf_iterator::equal` in favor of `operator==`.
- Deprecate `filesystem::equivalent` (operating on `filesystem::path` objects) in favor of an overload of `std::weak_equality`?